

playwright-agentic-kit

A guide to the code

Entry points, the most important functions, and a walkthrough of every part: the core fixtures, context files, agents, the healing guard, coverage and mutation testing, and the Supabase adapter.

Built 2026-10-08 from the repository. Code excerpts are read from the source at build time.

Contents

1. How to read this guide
 - The 30-second version
2. The repository at a glance
3. Entry points
 - Follow one test from start to finish
4. The core: config, adapter contract, fixtures
 - 4.1 `defineKitConfig()`
 - 4.2 The adapter contract
 - 4.3 `createKitTest()`: the fixtures
 - 4.4 Small helpers
 - 4.5 The project side: adapter, test object, specs
5. Context files
6. The agents
7. Healing, and the guard that keeps it honest
 - 7.1 The guard: `kit/heal/guard.ts`
 - 7.2 Snapshots: knowing exactly what the healer changed
 - 7.3 `npm run heal`: `kit/heal/local.ts`
 - 7.4 The report and publishing
8. Proving the tests: coverage and mutations
 - 8.1 Coverage: `kit/observe/coverage.ts`
 - 8.2 Mutations: `kit/observe/mutate.ts`
9. The Supabase adapter
 - 9.1 Keys and the local-only guard
 - 9.2 Users and sessions
 - 9.3 Database breaks
 - 9.4 The Supabase example
10. CI and repository hygiene
11. Explaining it: questions and answers
12. Glossary

1. How to read this guide

This guide walks through **playwright-agentic-kit**, a Playwright end-to-end testing template where AI agents plan, write and repair the tests, and deterministic code checks everything they do. It is written so you can learn the code from the top down and explain any part of it in detail.

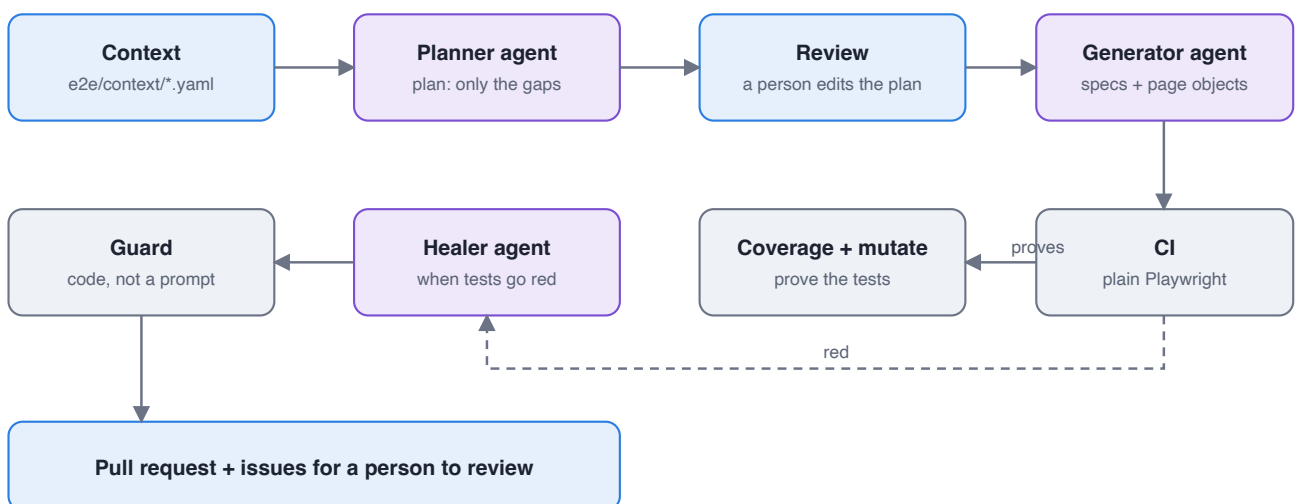
Suggested reading order: sections 2 and 3 give you the whole picture in fifteen minutes. Section 4 (the core) is the part every test touches; read it with the source open. Sections 5 to 9 are independent of each other, so read them in the order you need. Section 11 collects the questions people tend to ask, with answers.

Every code excerpt is taken from the repository when this PDF is built, with its file and line number. If an excerpt and the code ever disagree, the build fails, so the guide can't quietly go stale.

The 30-second version

Most "AI testing" either generates throwaway scripts or quietly self-heals tests at runtime, which hides real bugs. This kit takes the opposite stance:

- **Agents work at authoring time; CI stays deterministic.** The agents propose tests and repairs as ordinary code in a pull request. CI runs plain Playwright, with no model calls.
- **Agents are only as good as their context.** Each feature is described in a small, schema-validated YAML file: its rules, journeys and edge cases, each with an id. Every test names the ids it covers.
- **Rules are enforced by code, not trusted to a prompt.** A guard rejects any repair that weakens a test. A coverage report maps rules to tests. A mutation runner breaks the app on purpose to prove each test can fail.
- **The project-specific surface is tiny.** A new app writes one adapter (make a user, sign one in, seed data) and the context files. Supabase apps get a ready-made adapter.



The loop. Blue: people decide. Purple: agents propose. Grey: deterministic code checks.

2. The repository at a glance

The repository has two halves: `kit/` is the reusable framework and never mentions a specific app; `e2e/` is the project side, which a new project replaces. The demo app in `examples/` exists so the template runs on its own.

Path	Side	What it is
<code>kit/config.ts</code>	kit	<code>defineKitConfig()</code> : the Playwright configuration every project starts from
<code>kit/fixtures.ts</code>	kit	<code>createKitTest(adapter)</code> : the test fixtures (users, sign-in, data, two-person flows)
<code>kit/adapters/types.ts</code>	kit	The adapter contract: the only code a new project must write
<code>kit/adapters/supabase/</code>	kit	A ready-made adapter for Supabase apps, plus database breaks
<code>kit/context/</code>	kit	JSON schemas for context files and <code>check.ts</code> , the validator
<code>kit/heal/</code>	kit	The healing pipeline: guard, snapshots, report, <code>npm run heal</code> , CI publisher
<code>kit/observe/</code>	kit	Coverage report and mutation runner
<code>kit/unique.ts</code> , <code>kit/agents.ts</code>	kit	Unique test emails; handing seed values to agents
<code>.claude/agents/</code>	kit	The planner, generator and healer agent definitions
<code>e2e/context/</code>	project	<code>app.yaml</code> and one <code>*.feature.yaml</code> per feature
<code>e2e/support/</code>	project	<code>adapter.ts</code> (the app's adapter) and <code>test.ts</code> (the project's test)
<code>e2e/specs/</code> , <code>e2e/pages/</code>	project	Tests and page objects
<code>e2e/seeds/</code>	project	Known starting states for the agents; they also run as smoke tests
<code>e2e/plans/</code>	project	Test plans written by the planner and reviewed by a person
<code>e2e/mutations.ts</code>	project	Deliberate breaks of the app, each tied to the rule it violates
<code>examples/demo-app/</code>	demo	A dependency-free errands app with sign-in
<code>scripts/check-leaks.mjs</code>	repo	Fails CI if a private name appears in files or commit history

3. Entry points

Everything starts from one of these. Knowing which file each command runs is the fastest way into the code.

Command	Runs	Does
<code>npm test</code>	<code>playwright.config.ts</code> → <code>kit/config.ts</code>	The E2E suite, desktop and mobile
<code>npm run context:check</code>	<code>kit/context/check.ts</code>	Validates every context file against its schema
<code>npm run test:unit</code>	<code>node --test "kit/**/*.test.ts"</code>	Unit tests of the guard, report and Supabase keys
<code>npm run coverage</code>	<code>kit/observe/coverage.ts</code>	Which context items have tests
<code>npm run mutate</code>	<code>kit/observe/mutate.ts</code> + <code>e2e/mutations.ts</code>	Break the app on purpose; check the tests fail
<code>npm run plan -- <feature></code>	<code>kit/author/plan.ts</code>	Run the planner agent headlessly; only plans may change
<code>npm run generate -- <plan></code>	<code>kit/author/generate.ts</code>	Run the generator agent headlessly, behind the guard; run the new tests
<code>npm run heal</code>	<code>kit/heal/local.ts</code>	Run the healer agent locally, behind the guard
<code>npm run check</code>	several	Typecheck, format, context, unit tests, leak check
<code>npm run demo</code>	<code>examples/demo-app/server.mjs</code>	The demo app on port 4173
<code>npm run guide</code>	<code>docs/guide/build.mjs</code>	Rebuilds this PDF
Claude Code: "Use the e2e-planner agent..."	<code>.claude/agents/*.md</code> + <code>.mcp.json</code>	Plan, generate or heal tests
CI: push or pull request	<code>.github/workflows/ci.yml</code>	Jobs "Tests" and "Mutation check"
CI: failed run (opt-in)	<code>.github/workflows/heal.yml</code>	Heal in CI and open a pull request

Follow one test from start to finish

- `npx playwright test` loads `playwright.config.ts`, which calls `defineKitConfig()`. That sets the test folder (`e2e`), browsers, reporters and the web server that starts the app.
- Playwright finds `e2e/specs/errands.spec.ts`. It imports `test` from `e2e/support/test.ts`, which is `createKitTest(adapter)`: Playwright's `test` extended with the kit's fixtures.
- A test asks for `signedInPage`. Playwright builds the fixtures it depends on: `adapterCtx`, then `data`, `createdUsers`, `newUser`, `user` (which calls `adapter.createUser()`), and finally `signedInPage`, which calls `adapter.signIn()` on the page's browser context.

4. The test body runs against a page that is already signed in as a fresh, private user.
5. Teardown runs in reverse: the page closes, then `createdUsers` calls `adapter.deleteUser()` for every user the test made, and `data` calls `adapter.disposeData()`.
6. The reporters write `test-results/results.json` (read by the healer and mutation runner), `test-results/junit.xml` (read by the CI test report) and the HTML report.

4. The core: config, adapter contract, fixtures

4.1 defineKitConfig()

Every project's `playwright.config.ts` is a call to this function. It returns a normal Playwright config, with decisions baked in that make tests stable and agent-friendly.

```

export function defineKitConfig(options: KitConfigOptions): PlaywrightTestConfig {
  const ci = !!process.env.CI;
  const remote = process.env.E2E_BASE_URL;
  const baseUrl = remote ?? options.baseUrl;

  return defineConfig({
    testDir: options.testDir ?? './e2e',
    // The kit can live inside the test folder (e2e/kit). Its own unit tests use
    // node:test and are not Playwright tests.
    testIgnore: ['**/kit/**'],
    fullyParallel: true,
    forbidOnly: ci,
    // One retry in CI only, and the report marks retried tests as flaky.
    // Retries exist to collect a trace, not to make a red build green.
    retries: ci ? 1 : 0,
    reporter: [
      [ci ? 'github' : 'list'],
      ['html', { open: 'never', outputFolder: 'playwright-report' }],
      // Machine-readable results: the healer agent reads this to find what failed.
      ['json', { outputFile: 'test-results/results.json' }],
      // JUnit XML for CI test reports (e.g. dorny/test-reporter).
      ['junit', { outputFile: 'test-results/junit.xml', includeProjectInTestName: true }],
    ],
    use: {
      baseUrl,
      locale: options.locale ?? 'en-US',
      timezoneId: options.timezoneId ?? 'UTC',
      serviceWorkers: options.serviceWorkers ?? 'block',
      trace: 'retain-on-failure',
      screenshot: 'only-on-failure',
    },
    projects: [
      { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
      ...(options.mobile ? [{ name: 'mobile', use: { ...devices['Pixel 7'] } }] : []),
    ],
    webServer:
      options.webServer && !remote
        ? {
            command: options.webServer.command,
            url: baseUrl,
            env: options.webServer.env,
            timeout: options.webServer.timeout ?? 60_000,
            // E2E_FRESH_SERVER=1 forbids reusing a running server: the mutation runner
            // needs the app restarted with its changed code, never an old process.
            reuseExistingServer: !ci && !process.env.E2E_FRESH_SERVER,
          }
        : undefined,
    ...options.overrides,
  });
}

```

The decisions, and why:

- `E2E_BASE_URL` retargets a run (for example at staging) without editing the config. When it's set, no local web server starts.
- `testIgnore: ['**/kit/**']`: the kit can live inside the test folder (`e2e/kit`) in a real project. Its own unit tests use `node:test` and must not be loaded by Playwright.

- **Retries only in CI, and only one.** The report marks a retried test as flaky. A retry exists to collect a trace, not to turn red into green.
- **Three reporters.** A human one, `results.json` for machines (healer, mutation runner) and `junit.xml` for the CI report. `includeProjectInTestName` keeps desktop and mobile runs of a test apart.
- **Locale and timezone are pinned** (`en-US`, `UTC`), so text locators and dates don't change with the machine.
- **Service workers are blocked:** they cache aggressively and make runs depend on each other.
- `reuseExistingServer` is on locally for speed, but `E2E_FRESH_SERVER` turns it off. The mutation runner sets it, because a server kept alive from earlier would still be running unbroken code.

4.2 The adapter contract

This interface is the boundary between the kit and an app. Everything app-specific goes through it, which is why the rest of the kit never mentions any particular app.

kit/adapters/types.ts · line 4

```
export interface AdapterContext {
  /** Request context bound to `baseUrl`, independent of any browser page. */
  request: APIRequestContext;
  baseUrl: string;
  testInfo: TestInfo;
}
```

kit/adapters/types.ts · line 19

```
export interface KitAdapter<TUser, TData> {
  /** App-specific seeding helpers, exposed to tests as the `data` fixture. */
  data(ctx: AdapterContext): TData | Promise<TData>;
  /** Create a brand-new user that no other test can see. */
  createUser(ctx: AdapterContext, data: TData): Promise<TUser>;
  /** Make `context` signed in as `user`. Prefer an API call over clicking through the login form. */
  signIn(context: BrowserContext, user: TUser, ctx: AdapterContext): Promise<void>;
  /** Remove what `createUser` made. Runs after each test, pass or fail. */
  deleteUser?(user: TUser, ctx: AdapterContext): Promise<void>;
  /** Clean up whatever the `data` helpers created during the test. Runs after each test, pass or fail. */
  disposeData?(data: TData, ctx: AdapterContext): Promise<void>;
}
```

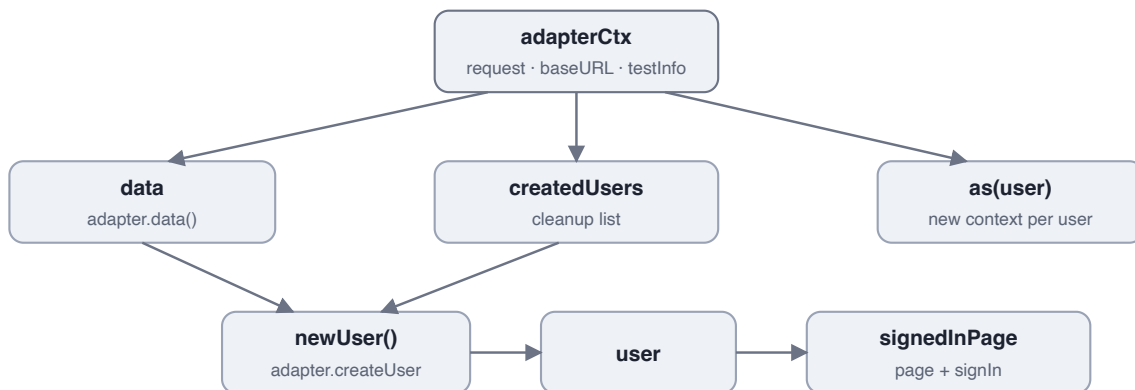
Method	Called by	When
<code>data(ctx)</code>	<code>data</code> fixture	Once per test that uses <code>data</code> (or anything built on it)
<code>createUser(ctx, data)</code>	<code>newUser()</code> / <code>user</code>	Each time a test needs a fresh user
<code>signIn(context, user, ctx)</code>	<code>signInPage</code> , <code>signIn()</code> , <code>as()</code>	Before the page is used

Method	Called by	When
<code>deleteUser(user, ctx)</code>	<code>createdUsers</code> teardown	After the test, pass or fail
<code>disposeData(data, ctx)</code>	<code>data</code> teardown	After the test, for users the data helpers made

Explain it like this: "The adapter is a contract with five methods. A new app implements how to make a user, sign one in and clean up; the kit turns that into fixtures every test can use. That boundary is why the framework stays reusable."

4.3 createKitTest(): the fixtures

A Playwright *fixture* is a value a test asks for by name. Its code runs *setup* before `await use(value)`, the test runs during `use`, and *teardown* runs after it. Fixtures can depend on other fixtures. That's all you need to read this file.



Fixture dependencies in `createKitTest()`. An arrow means "is built from". Teardown runs in reverse: users are deleted after the test that used them.

kit/fixtures.ts · line 4

```
export interface KitFixtures<TUser, TData> {  
  /** App-specific seeding helpers from the adapter. */  
  data: TData;  
  /** A fresh user created for this test only. Not signed in until you use `signInPage` or `as`. */  
  user: TUser;  
  /** A page whose browser context is already signed in as `user`. */  
  signInPage: Page;  
  /**  
   * Open a page signed in as any user, each in its own browser context.  
   * Use it for flows between accounts, like messaging or sharing.  
   */  
  as: (user: TUser) => Promise<Page>;  
  /** Make another fresh user, cleaned up after the test like `user`. */  
  newUser: () => Promise<TUser>;  
  /**  
   * Sign the default `page` in as a specific user, e.g. one made with `data` helpers.  
   * Agent seed files use this, because the agents take over the default `page`.  
   */  
  signIn: (user: TUser) => Promise<void>;  
}
```

The first fixtures: the context handed to every adapter method, the data helpers, and the cleanup list.

kit/fixtures.ts · line 33

```
adapterCtx: async ({ request, baseUrl }, use, testInfo) => {  
  if (!baseUrl) throw new Error('baseUrl is not set. Configure it with defineKitConfig().');  
  await use({ request, baseUrl, testInfo });  
},
```

kit/fixtures.ts · line 44 · cleanup tries every user even if one fails

```
createdUsers: async ({ adapterCtx }, use) => {  
  const created: TUser[] = [];  
  await use(created);  
  if (adapter.deleteUser) {  
    // Try every user even if one fails, so a single error doesn't leave the rest behind.  
    const errors: unknown[] = [];  
    for (const u of created) await adapter.deleteUser(u, adapterCtx).catch((e: unknown) => errors.push(e));  
    if (errors.length) throw new AggregateError(errors, `Cleaning up ${errors.length} test user(s) failed`);  
  }  
},
```

kit/fixtures.ts · line 55

```
newUser: async ({ adapterCtx, data, createdUsers }, use) => {  
  await use(async () => {  
    const u = await adapter.createUser(adapterCtx, data);  
    createdUsers.push(u);  
    return u;  
  });  
},
```

`user` is simply `await newUser()`. `signInPage` signs the built-in `page`'s context in as `user` ;
`signIn(user)` does the same for any user (seeds use it, because the agents take over the default page).

kit/fixtures.ts · line 109

```
signInPage: async ({ page, user, adapterCtx }, use) => {  
  await adapter.signIn(page.context(), user, adapterCtx);  
  await use(page);  
},
```

`as(user)` is the two-person fixture. Each user gets their own browser context, which means their own cookies and storage, so two people can be signed in at once. It copies the project's device options so both users see the same phone or desktop.

```

as: async (
  {
    browser,
    adapterCtx,
    baseUrl,
    locale,
    timezoneId,
    serviceWorkers,
    viewport,
    userAgent,
    deviceScaleFactor,
    isMobile,
    hasTouch,
  },
  use,
) => {
  const contexts: Awaited<ReturnType<typeof browser.newContext>>[] = [];
  // Same options the built-in `page` gets, so every user sees the same device and locale.
  const options = {
    baseUrl,
    locale,
    timezoneId,
    serviceWorkers,
    viewport,
    userAgent,
    deviceScaleFactor,
    isMobile,
    hasTouch,
  };
  await use(async (u) => {
    const context = await browser.newContext(options);
    contexts.push(context);
    await adapter.signIn(context, u, adapterCtx);
    return context.newPage();
  });
  for (const c of contexts) await c.close();
},

```

Why a separate `createdUsers` fixture? Teardown order. Because `newUser` depends on `createdUsers`, Playwright tears `createdUsers` down *after* every page and context that used those users, so a user is never deleted while a page still needs them.

4.4 Small helpers

```

export function uniqueEmail(testInfo: TestInfo, domain = 'example.test'): string {
  const stamp = Date.now().toString(36);
  return `e2e-w${testInfo.workerIndex}-r${testInfo.retry}-${stamp}-${++counter}@${domain}`;
}

```

Worker index, retry number, a timestamp and a counter make the address unique even with parallel workers and retries against one shared backend. The `e2e-` prefix is what the stale-user sweep looks for.

kit/agents.ts · line 12

```
export async function exposeToAgents(page: Page, values: Record<string, unknown>): Promise<void> {
  await page.addInitScript((v) => {
    (window as unknown as { __e2e: unknown }).__e2e = v;
  }, values);
}
```

The planner and generator explore from a seed's page, but they can't see its fixtures. A seed that creates an account hands the credentials over on `window.__e2e`; the agents read them with their browser tool. An init script survives navigation, so the values stay available as the agent moves around.

4.5 The project side: adapter, test object, specs

The demo app's adapter shows the contract in its simplest form: test-only endpoints create users, and sign-in goes through the API, not the form.

e2e/support/adapter.ts · line 24

```
export const adapter = defineAdapter<User, ErrandsData>({
  data: (ctx) => ({
    userWithErrands: (titles) => createUser(ctx, titles),
  }),

  createUser: (ctx) => createUser(ctx),

  async signIn(context, user) {
    // Through the API, not the form: faster, and a broken login form then only
    // fails the tests that are about logging in.
    const res = await context.request.post('/api/session', { data: { email: user.email, password:
user.password } });
    if (!res.ok()) throw new Error(`Sign-in failed for ${user.email}: ${res.status()}`);
  },

  async deleteUser(user, ctx) {
    await ctx.request.delete('/__test__/users', { data: { email: user.email } });
  },
});
```

e2e/support/test.ts · first 6 lines

```
import { createKitTest } from '../..//kit';
import { adapter } from '../adapter';

/** Specs import `test` and `expect` from here, never from @playwright/test directly. */
export const test = createKitTest(adapter);
export { expect } from '../..//kit';
```

A spec reads like a normal Playwright test. The `covers` annotation links it to context ids; that link is what the coverage report and the mutation runner rely on.

```
'completing an errand updates the count and survives a reload',
{
  annotation: [
    { type: 'covers', description: 'errands#complete-errand' },
    { type: 'covers', description: 'errands#done-persists' },
  ],
},
async ({ page, data, as }) => {
  const owner = await data.userWithErrands(['Buy milk', 'Post parcel']);
  page = await as(owner);
  const errands = new ErrandsPage(page);
  await errands.goto();

  await errands.checkbox('Buy milk').check();
  await expect(errands.remaining).toHaveText('1 errand left');

  await page.reload();
  await expect(errands.checkbox('Buy milk')).toBeChecked();
  await expect(errands.checkbox('Post parcel')).not.toBeChecked();
},
);
```

5. Context files

Context is the source of truth for what the app *should* do. People write it; agents read it and never edit it. It is structured YAML with a schema, so it can be validated, versioned with the code and fed to agents without prompt sprawl.

e2e/context/errands.feature.yaml · first 25 lines · a feature context

```
# yaml-language-server: $schema=../../kit/context/feature.schema.json
feature: errands
summary: Signed-in users add, complete and delete errands on a private list.
risk: high
entry: /#/errands
personas: [member]
rules:
  - id: private-lists
    rule: A user only ever sees their own errands
  - id: no-blank-titles
    rule: Blank titles are not saved
  - id: remaining-counter
    rule: The remaining counter counts errands that are not done, with correct singular and plural
  - id: done-persists
    rule: Completed state persists across reloads
journeys:
  - id: add-errand
    name: Add an errand
    priority: critical
    persona: member
    steps:
      - Type a title into "New errand"
      - Press Add
    expect:
      - The errand appears in the list, unchecked
```

- **Every rule, journey and edge case has an id.** A test names what it covers as `feature#id` (or `feature#edge:id`), so coverage can be traced in both directions.
- `risk` tells the planner how deep to go: high means every rule, journey and edge case, with negative paths.
- `known_issues` lists real bugs that are accepted for now, so the healer neither reports them again nor "heals" around them.
- `app.yaml` holds project-wide conventions: locator rules, how tests get data, things never to do, and what can't be automated.

```
for (const file of files) {  
  const doc = parse(await readFile(join(dir, file), 'utf8'));  
  const validate = file === 'app.yaml' ? validateApp : validateFeature;  
  if (validate(doc)) {  
    console.log(`✓ ${file}`);  
    continue;  
  }  
  failed++;  
  console.error(`✗ ${file}`);  
  for (const err of validate.errors ?? []) console.error(`    ${err.instancePath || '/'}  
    ${err.message}`);  
}
```

6. The agents

An agent is a Markdown file in `.claude/agents/`: a short header (name, description, the exact tools it may use, model) and instructions. Claude Code runs it; the Playwright MCP server in `.mcp.json` gives it a real browser and the ability to run tests. They started from Playwright's own `init-agents` templates and were rewritten around the context files.

Agent	Reads	Writes	Key constraint
e2e-planner	the feature context, existing <code>covers</code> annotations, the live app	<code>e2e/plans/<feature>.plan.md</code>	Plans only gaps. If the app contradicts a rule, it records a <i>finding</i> instead of planning a test around a bug.
e2e-generator	a plan someone has reviewed, page objects, fixtures	tests in <code>e2e/specs/</code> , page-object methods	Never bends an expectation to match the app; a scenario that fails is reported as blocked.
e2e-healer	failing tests, <code>results.json</code> , the context	fixes, <code>.heal/report.json</code>	Classifies every failure first. App bugs and stale context are reported, never "healed".

Running them. In a Claude Code session, ask for an agent by name. From a terminal, `npm run plan`, `generate` and `heal` run them headlessly through the `claude` CLI with your own login (`kit/claude.ts`). All three snapshot the working tree first and check what the agent changed afterwards: the planner may only write plans, and the generator and healer go through the guard (section 7), which undoes all their changes on a violation.

Seeds (`e2e/seeds/`) are tiny tests that put the browser in a known state: signed out, signed in with an empty list, signed in with data. An agent starts from a seed, and because seeds also run as smoke tests, a broken seed fails CI instead of confusing an agent.

The healer's classification is the heart of honest healing:

Class	Meaning	What the healer does
selector	Same control, same purpose, different name or structure	Updates the locator, in the page object
timing	The assertion ran before the UI settled	Uses a web-first assertion on the awaited state; never a fixed wait
test-bug	The test contradicts the context	Fixes the test to match the context
data	Seeding or fixture problem	Fixes the adapter or fixture usage
environment	Server down, browser missing	Changes nothing; reports it
app-bug	The app breaks a rule in the context	Changes nothing; reports it with the rule quoted

Class	Meaning	What the healer does
context-drift	The app changed on purpose; the context is stale	Changes nothing; reports the stale entry

Explain it like this: "A failing test is evidence. The healer only repairs failures where the test is wrong and the app is right. Everything else becomes a report. And because a prompt can be ignored, code checks every change it makes."


```

export function checkDiff(files: FileChange[], options: GuardOptions = { allowed: DEFAULT_ALLOWED }): GuardResult {
  const violations: string[] = [];
  const warnings: string[] = [];

  for (const f of files) {
    if (!options.allowed.some((p) => f.path.startsWith(p))) {
      violations.push(`${f.path}: outside the agents' area (${options.allowed.join(', ')} )`);
      continue;
    }
    if (f.status === 'deleted') {
      violations.push(`${f.path}: deletes a file`);
      continue;
    }

    for (const line of f.added) {
      for (const [re, why] of FORBIDDEN) {
        if (re.test(line)) violations.push(`${f.path}: ${why}: \`${line.trim()}\``);
      }
    }

    const lost = count(f.removed, EXPECT) - count(f.added, EXPECT);
    if (lost > 0) violations.push(`${f.path}: removes ${lost} assertion(s)`);

    // Each kind of check that disappears must come back. Swapping a one-off
    // value check (toBe) for a web-first one (toHaveText) is a strengthening.
    const removedMatchers = collect(f.removed, MATCHER);
    const addedMatchers = collect(f.added, MATCHER);
    let spareWebFirst = minus(addedMatchers, removedMatchers).filter((m) =>
!NON_WEB_FIRST.has(m)).length;
    for (const m of minus(removedMatchers, addedMatchers)) {
      if (NON_WEB_FIRST.has(m) && spareWebFirst > 0) {
        spareWebFirst--;
        continue;
      }
      violations.push(`${f.path}: replaces or drops a ${m}() check`);
    }

    // Exact expected value → regex is the classic way to make a test stop caring.
    const regexAdded = collect(f.added, REGEX_ARG);
    for (const m of new Set(collect(f.removed, LITERAL_ARG))) {
      if (regexAdded.includes(m)) violations.push(`${f.path}: loosens ${m}() from an exact value to
a pattern`);
    }

    // A changed expected value is only legitimate when the old one contradicted the
    // context file (class test-bug). Code can't judge that, so a human must.
    const before = [...f.removed.join('\n').matchAll(LITERAL_ARG)].map((m) => `${m[1]}:${m[3]}`);
    const after = [...f.added.join('\n').matchAll(LITERAL_ARG)].map((m) => `${m[1]}:${m[3]}`);
    for (const changed of minus(before, after)) {
      const [matcher, value] = changed.split(/:(.*)/s);
      if (after.some((a) => a.startsWith(`${matcher}:`))) {
        warnings.push(`${f.path}: changes the expected value of ${matcher}() (was "${value}")`);
      }
    }
  }

  return { violations, warnings };
}

```

The checks, in order: **only allowed folders** (`e2e/specs` , `pages` , `support` ; never `context`, `app code` or `workflows`); **no deleted files**; **no forbidden patterns** on added lines (`skips` , `.only` , fixed waits, swallowed errors, commented-out assertions); **no fewer** `expect(` `calls`; **no matcher may disappear**, except a one-off `toBe` swapped for a web-first matcher, which is stronger; **no exact value loosened to a regex**. A changed expected value is only a *warning*, because fixing a genuinely wrong test can need one, and only a person can judge that.

Why line-based? It is a safety net against the usual ways of making a red test green dishonestly, not a full TypeScript analysis. Simple rules are predictable, explainable and fast to test.

7.2 Snapshots: knowing exactly what the healer changed

A developer often heals with uncommitted work of their own in the tree. Comparing against `HEAD` would blame that work on the healer. So the kit snapshots the whole working tree as a git tree object before the healer runs, and again after, and diffs the two.

kit/heal/snapshot.ts · line 12

```
export function snapshotTree(): string {
  const dir = mkdtempSync(join(tmpdir(), 'heal-index-'));
  const env = { ...process.env, GIT_INDEX_FILE: join(dir, 'index') };
  try {
    execFileSync('git', ['read-tree', 'HEAD'], { env });
    execFileSync('git', ['add', '--all'], { env });
    return execFileSync('git', ['write-tree'], { env, encoding: 'utf8' }).trim();
  } finally {
    rmSync(dir, { recursive: true, force: true });
  }
}
```

The trick is `GIT_INDEX_FILE` : git uses a temporary index, so `git add --all` captures every file (respecting `.gitignore`) without touching the developer's real staging area or any branch. `restoreFrom()` uses the same snapshot to undo the healer's changes when the guard rejects them.

7.3 npm run heal: kit/heal/local.ts

kit/heal/local.ts · line 39

```
const before = snapshotTree();
rmSync('.heal', { recursive: true, force: true });
mkdirSync('.heal');

step(3, `Running the e2e-healer agent (max ${config.maxTurns} turns${config.model ? `, model ${config.model}` : ''})`);
const healerStatus = runAgent(config, {
  agent: 'e2e-healer',
  prompt:
    'Heal the tests that fail in test-results/results.json. Write .heal/report.json as your instructions describe. Do not commit.',
  tools: 'Read,Glob,Grep,LS,Edit,Write,mcp__playwright-test__',
  maxTurns: config.maxTurns,
  model: config.model || undefined,
});
if (healerStatus !== 0) console.warn(`\n\nThe healer exited with code ${healerStatus}. Checking what it left behind.`);

step(4, 'Guarding its changes');
const changes = diffTrees(before, snapshotTree());
const guard = checkDiff(changes);
for (const f of changes) console.log(`  changed: ${f.path}`);
for (const w of guard.warnings) console.log(`  ! ${w}`);
if (guard.violations.length) {
  for (const v of guard.violations) console.log(`  x ${v}`);
  restoreFrom(before, changes);
  console.log(
    '\nRejected. The healer broke the rules above, so its changes have been undone. Your own work is untouched.',
  );
  process.exit(1);
}
console.log(`  guard passed (${changes.length} file(s))`);
```

The healer runs as a headless Claude Code session (`--agent e2e-healer`) with a turn limit, using the developer's own login. Settings come from a git-ignored `heal.local.json` (`kit/heal/config.ts`): where Claude Code is, model, turn limit, whether to commit the repairs, whether to file issues.

7.4 The report and publishing

The healer writes a machine-readable report; everything after it reads that, not the agent's chat output.

```
export interface HealReport {
  healed: { test: string; class: HealedClass; change: string; evidence: string }[];
  escalated: {
    test: string;
    class: EscalatedClass;
    covers?: string;
    expected: string;
    actual: string;
    evidence: string;
  }[];
  /**
   * Notes for a person: context entries that look stale or incomplete, noticed while
   * healing (e.g. a journey still says "Press Add" after a rename). Never acted on by the agent.
   */
  context_suggestions?: { context: string; suggestion: string }[];
}
```

In CI, `.github/workflows/heal.yml` runs when CI fails (only if the `HEAL_IN_CI` repository variable is `true`, never for `heal/` branches, never for forks). It reproduces the failure, runs the healer, the guard and the suite, then `kit/heal/publish.ts` opens a `heal/<run>` pull request and `kit/heal/github.ts` files deduplicated issues for app bugs and stale context.

8. Proving the tests: coverage and mutations

AI-written tests are only worth something if they test the right things and can actually fail. Two tools check both. They share one helper that reads the tests without running them.

kit/observe/tests.ts · line 29

```
export function listTests(): ListedTest[] {
  // Read the JSON from a file, not stdout: anything else printing (a web server,
  // a stray console.log in a spec) would corrupt it.
  const dir = mkdtempSync(join(tmpdir(), 'pw-list-'));
  const file = join(dir, 'list.json');
  try {
    execFileSync('npx', ['playwright', 'test', '--list', '--reporter=json'], {
      stdio: 'ignore',
      env: { ...process.env, PLAYWRIGHT_JSON_OUTPUT_FILE: file },
    });
  } catch {
    // A non-zero exit still writes the report when listing works; checked below.
  }
  if (!existsSync(file)) throw new Error('Listing the tests failed. Run `npx playwright test --list` to see why.');
```

```
  const report = JSON.parse(readFileSync(file, 'utf8')) as { config: { rootDir: string }; suites: Suite[] };
  rmSync(dir, { recursive: true, force: true });
  const out: ListedTest[] = [];
  const walk = (suite: Suite, file: string | undefined) => {
    for (const spec of suite.specs ?? []) {
      for (const t of spec.tests) {
        const covers = t.annotations.filter((a) => a.type === 'covers' && a.description).map((a) => a.description!);
        const where = t.annotations.find((a) => a.location)?.location?.file ?? file ?? '';
        out.push({
          file: relative(process.cwd(), where),
          line: spec.line,
          title: spec.title,
          project: t.projectName,
          covers,
        });
      }
    }
    for (const child of suite.suites ?? []) walk(child, file);
  };
  for (const suite of report.suites) walk(suite, `${report.config.rootDir}/${(suite as { file?: string }).file ?? ''}`);
  return out;
}
```

Playwright's list mode (`--list --reporter=json`) returns every test with its file, line, browser project and annotations. The JSON goes to a file rather than stdout, so stray output can't corrupt it. `uniqueTests()` collapses the desktop and mobile copies of a test, keyed by file, line *and* title, because tests generated in a loop share a line.

8.1 Coverage: kit/observe/coverage.ts

It loads every feature context, builds the list of items (rules, journeys, edge cases), and maps each to the tests whose `covers` annotation names it.

kit/observe/coverage.ts · line 60

```
const tests = uniqueTests(listTests());
const byId = new Map<string, ListedTest[]>();
for (const t of tests) for (const c of t.covers) byId.set(c, [...(byId.get(c) ?? []), t]);

const known = new Set(items.map((i) => i.id));
const unknown = [...byId.entries()].filter(([id]) => !known.has(id));
const missing = items.filter((i) => !byId.has(i.id));
const missingRequired = missing.filter((i) => i.required);
const covered = items.length - missing.length;
const pct = items.length ? Math.round((covered / items.length) * 100) : 100;
const untagged = tests.filter((t) => t.covers.length === 0 && !t.file.includes('/seeds/'));
```

With `--strict` (used in CI) it fails on an id that matches nothing, usually a typo or a renamed id, and on any rule or critical journey without a test. Optional items are reported but don't fail the build.

8.2 Mutations: kit/observe/mutate.ts



`npm run mutate (kit/observe/mutate.ts)`, once per break in `e2e/mutations.ts`.

A project declares its breaks in `e2e/mutations.ts`. Each names the context id it violates, and either text edits or `apply()` / `undo()` functions.

kit/observe/mutations.ts · line 24

```
export interface Mutation {  
  /** Short, unique slug. */  
  id: string;  
  /** Context id(s) this break violates. Tests covering any of them are run. */  
  covers: string | string[];  
  /** What the break does, in plain words: shown in the report. */  
  description: string;  
  /** Text edits to app files. Restored from a journal even if the run is interrupted. */  
  edits?: Edit[];  
  /** For breaks outside files (a database, a feature flag): change it here... */  
  apply?: () => void | Promise<void>;  
  /** ...and put it back here. Called even when the tests fail or crash. */  
  undo?: () => void | Promise<void>;  
}
```

The safety net comes first. Original file contents are written to a journal before any edit, and restored after every break, on Ctrl-C, and at the start of the next run if a run was killed.

kit/observe/mutate.ts · line 31

```
function restoreJournal(): string[] {  
  if (!existsSync(JOURNAL)) return [];  
  const originals = JSON.parse(readFileSync(JOURNAL, 'utf8')) as Record<string, string>;  
  for (const [file, content] of Object.entries(originals)) writeFileSync(file, content);  
  rmSync(JOURNAL);  
  return Object.keys(originals);  
}
```

Running the covering tests. Note the two refusals at the end: a run that produced no results, or where nothing ran (for example, the app server couldn't start), throws, so it can never be mistaken for "all passed".

```

function run(selected: ListedTest[]): string[] {
  mkdirSync('.mutate', { recursive: true });
  rmSync(RESULTS, { force: true });
  spawnSync(
    'npx',
    [
      'playwright',
      'test',
      ...selected.map((t) => `${t.file}:${t.line}`),
      `--project=${project}`,
      '--reporter=json',
      '--retries=0',
    ],
    { stdio: 'ignore', env: { ...process.env, PLAYWRIGHT_JSON_OUTPUT_FILE: RESULTS,
E2E_FRESH_SERVER: '1' } },
  );
  if (!existsSync(RESULTS)) throw new Error('The test run produced no results.');
```

const report = JSON.parse(readFileSync(RESULTS, 'utf8')) as PwResult;

const failed: string[] = [];

let ran = 0;

const walk = (s: Suite) => {

for (**const** spec **of** s.specs ?? []) {

 ran += spec.tests.length;

const bad = spec.tests.filter((t) => t.status === 'unexpected');

if (!bad.length) **continue**;

// A test timeout (not an assertion's) may be the break, or just a slow machine.

const timedOut = bad.every((t) => t.results?.some((r) => r.status === 'timedOut'));

 failed.push(timedOut ? `\${spec.title} (timed out)` : spec.title);

 }

for (**const** c **of** s.suites ?? []) walk(c);

};

for (**const** s **of** report.suites) walk(s);

const errors = (report.errors ?? []).map((e) => e.message?.split('\n')[0]).filter(Boolean);

if (errors.length || ran === 0) **throw new** Error(errors[0] ?? 'No tests ran.');

return failed;

}

The main loop, one break at a time. Every edit is checked first (it must occur exactly once, or the break is *stale*), then applied, the covering tests run, and the result is classified. The `finally` block always restores.

```

for (const m of mutations) {
  const selected = testsFor(m);
  const base = {
    id: m.id,
    description: m.description,
    covers: coversOf(m),
    tests: selected.length,
    failed: [] as string[],
  };
  if (!selected.length) {
    results.push({ ...base, status: 'no tests', note: 'no test covers these ids' });
    continue;
  }

  // Check every edit still applies before touching anything.
  const originals: Record<string, string> = {};
  let stale: string | undefined;
  for (const e of m.edits ?? []) {
    const content = originals[e.file] ?? (existsSync(e.file) ? readFileSync(e.file, 'utf8') :
undefined);
    const count = content?.split(e.find).length ?? 0;
    if (content === undefined) stale = `${e.file} doesn't exist`;
    else if (count - 1 !== 1) stale = `${e.find.slice(0, 40)}" occurs ${count - 1} times in
${e.file}, expected once`;
    if (stale) break;
    originals[e.file] ??= content!;
  }
  if (stale) {
    results.push({ ...base, status: 'stale', note: stale });
    continue;
  }

  process.stdout.write(`${m.id} ... `);
  mkdirSync('.mutate', { recursive: true });
  writeFileSync(JOURNAL, JSON.stringify(originals));
  try {
    const current = { ...originals };
    for (const e of m.edits ?? []) {
      current[e.file] = current[e.file].replace(e.find, e.replace);
      writeFileSync(e.file, current[e.file]);
    }
    pendingUndo = m.undo;
    try {
      // A break that can't be applied (its target changed) is stale, not a crash.
      try {
        await m.apply?();
      } catch (e) {
        results.push({ ...base, status: 'stale', note: `apply() failed: ${(e as
Error).message.split('\n')[0]}` });
        console.log('STALE');
        continue;
      }
    }
    const failed = run(selected);
    const status: Status = failed.length ? 'caught' : 'survived';
    const onlyTimeouts = failed.length > 0 && failed.every((f) => f.endsWith('(timed out)'));
    results.push({
      ...base,
      status,
      failed,
    });
  }
}

```

```

    note: onlyTimeouts ? 'caught only by test timeouts: check it is the break, not a slow run'
: undefined,
});
console.log(status === 'caught' ? `caught by ${failed.length}` : 'SURVIVED');
} catch (e) {
  // The run broke (server didn't start, nothing ran): unknown, never "survived".
  results.push({ ...base, status: 'error', note: (e as Error).message });
  console.log(`ERROR: ${(e as Error).message}`);
}
} finally {
  await m.undo?();
  pendingUndo = undefined;
  restoreJournal();
}
}

```

Status	Meaning
caught	At least one covering test failed with the app broken. Flagged if only test timeouts caught it, since a slow run looks the same.
survived	Every covering test passed with the app broken: the tests are too weak.
no tests	Nothing covers the break's ids.
stale	The break no longer applies: its text changed or appears twice, or <code>apply()</code> failed.
error	The run itself failed. Unknown; never counted as caught or survived.

A real catch Its first run found a weak test written by hand: "a blank title adds nothing" checked an empty list that was already empty before the server answered, so it passed even when the server saved blank titles. The fix: wait for the server's answer and reload before asserting.

9. The Supabase adapter

`kit/adapters/supabase/` is a ready-made adapter for apps on Supabase Auth with `supabase-js` in the browser. An app only writes its own parts: how to finish a new account (`setUp`) and its own helpers (`data`).

`kit/adapters/supabase/index.ts` · line 69

```
export interface SupabaseAdapterOptions<TUser extends SupabaseAccount, TOptions, TExtra extends object> {
  /**
   * Finish a fresh, confirmed account the way the app's signup would: profile rows, an
   * onboarding RPC. Return the fields your user type adds to the account.
   */
  setUp?: (
    account: SupabaseAccount,
    options: TOptions | undefined,
    tools: { apiAs: SupabaseHelpers<TUser, TOptions>['apiAs'] },
  ) => Promise<Omit<TUser, keyof SupabaseAccount>>;
  /** App-specific helpers for the `data` fixture, built on the generic ones. */
  data?: (helpers: SupabaseHelpers<TUser, TOptions>, ctx: AdapterContext) => TExtra;
  /** Password for every test user. Must satisfy the project's password rules. */
  password?: string;
  /** Hosts besides localhost that may be used. Only for disposable environments. */
  allowedHosts?: string[];
}
```

9.1 Keys and the local-only guard

`kit/adapters/supabase/keys.ts` · line 19

```
export function resolveKeys(
  env: Record<string, string | undefined>,
  status: () => Record<string, string>,
  allowedHosts: string[] = LOCAL_HOSTS,
): SupabaseKeys {
  let { E2E_SUPABASE_URL: url, E2E_SUPABASE_ANON_KEY: anonKey, E2E_SUPABASE_SERVICE_ROLE_KEY: serviceRoleKey } = env;
  if (!url || !anonKey || !serviceRoleKey) {
    const s = status();
    url ??= s.API_URL;
    anonKey ??= s.ANON_KEY;
    serviceRoleKey ??= s.SERVICE_ROLE_KEY;
  }
  if (!url || !anonKey || !serviceRoleKey) throw new Error('Supabase URL or keys missing.');
```

```
  const host = new URL(url).hostname;
  if (!allowedHosts.includes(host)) {
    throw new Error(
      `E2E tests create and delete accounts, so they only run against local Supabase
      (${allowedHosts.join(', ')}), not ${host}.`,
    );
  }
  return { url, anonKey, serviceRoleKey };
}
```

Tests create and delete accounts, so any host but localhost is refused. `resolveKeys` takes its inputs as parameters (environment and a status function), which is what makes it unit-testable without a running Supabase.

9.2 Users and sessions

kit/adapters/supabase/index.ts · line 134

```
async function newUser(ctx: AdapterContext, opts?: TOptions): Promise<TUser> {
  const account: SupabaseAccount = { id: '', email: uniqueEmail(ctx.testInfo), password };
  const r = await request(ctx, 'POST', '/auth/v1/admin/users', keys().serviceRoleKey, {
    email: account.email,
    password,
    email_confirm: true,
  });
  if (!r.ok) throw new Error(`Creating test user failed: ${r.status} ${JSON.stringify(r.body)}`);
  account.id = (r.body as { id: string }).id;
  const extra = options.setUp
    ? await options.setUp(account, opts, { apiAs: (u, m, p, b) => apiAs(ctx, u, m, p, b) })
    : {};
  return { ...account, ...extra } as TUser;
}
```

Users are created confirmed through the Auth admin API (service-role key), then handed to the app's `setUp`, for example to call an onboarding RPC the way the real signup form does.

kit/adapters/supabase/index.ts · line 190

```
async signIn(context, user, ctx) {
  const session = await passwordGrant(ctx, user);
  await context.setStorageState({
    cookies: [],
    origins: [
      {
        origin: new URL(ctx.baseURL).origin,
        localStorage: [{ name: storageKey(keys().url), value: JSON.stringify(session) }],
      },
    ],
  });
},
```

Sign-in gets a real session with a password grant and puts it where supabase-js keeps it, under the same storage key supabase-js derives from the URL. `setStorageState` sets it once; an init script would restore it on every page load and quietly undo sign-out in tests.

The `data` fixture also gets helpers for checking below the UI: `apiAs(user, method, path, body)` calls the API as that user, to prove what row-level security and database functions allow; `canSignIn(user)` says whether an account still exists; `sessionIn(page)` and `refreshTokenWorks(token)` prove a sign-out ended the session on the server. Checking a refresh token uses it up (Supabase rotates them), so check only after the action under test.

9.3 Database breaks

kit/adapters/supabase/db.ts · line 41

```
export function dbFunction(name: string, find: string, replace: string): Part {
  let original = '';
  return {
    apply() {
      original = query(`select pg_get_functiondef('public.${name} '::regproc)`);
      const count = original.split(find).length - 1;
      if (count !== 1) throw new Error(`'${find.slice(0, 40)}' occurs ${count} times in ${name}(),
expected once`);
      psql(original.replace(find, replace) + ';');
    },
    undo() {
      if (original) psql(original + ';');
    },
  };
}
```

`dbSql(apply, undo)` covers anything else, such as replacing a policy with a looser one. Each helper reads the object's real definition from the local database at the moment it applies, and restores exactly that, so it never works from a stale copy. `db()` combines several breaks, undone in reverse; that's how a rule enforced twice (for example a function check plus a table constraint) is broken in both places at once.

9.4 The Supabase example

`examples/supabase-app` runs the adapter against a real local Supabase stack in this repository's own CI: a notes app whose rules live in the database. Its whole adapter:

examples/supabase-app/e2e/support/adapter.ts · line 28

```
export const adapter = supabaseAdapter<User, NewUserOptions, NotesHelpers>({
  async setUp(account, options = {}, { apiAs }) {
    const username = `e2e_${randomBytes(6).toString('hex')}`;
    if (options.onboarded !== false) {
      const r = await apiAs(account, 'POST', '/rest/v1/rpc/complete_profile', { chosen_username:
username });
      if (r.status >= 300) throw new Error(`complete_profile failed: ${r.status}
${JSON.stringify(r.body)}`);
    }
    return { username };
  },

  data: ({ apiAs }) => ({
    async addNotes(user, ...bodies) {
      for (const body of bodies) {
        const r = await apiAs(user, 'POST', '/rest/v1/notes', { body });
        if (r.status >= 300) throw new Error(`Adding a note failed: ${r.status}
${JSON.stringify(r.body)}`);
      }
    },
  }),
});
```

Row-level security, twice A break that only loosened the delete policy survived: in Postgres a `DELETE ... WHERE` must also pass the read policy, because the `WHERE` clause reads the row. The rule was enforced twice, so the break has to loosen both. When a break survives, look for a second layer before blaming the test.

10. CI and repository hygiene

Job / step	What it proves
Typecheck, format and context	The code compiles, follows Prettier, and every context file matches its schema
Unit tests	The guard, report and Supabase key handling; results published as a check
Leak check	No private name (from a secret denylist) in files or commit history
E2E tests	The whole suite, desktop and mobile; results published as a check with failures annotated
Context coverage (strict)	No broken covers ids; every rule and critical journey has a test
Mutation check (own job)	Every declared break is caught

Test results are published with `dorny/test-reporter` from the JUnit files, as check runs on the commit or pull request. That needs `checks: write` on the job and `use-actions-summary: false`, otherwise version 3 writes to the job summary instead.

scripts/check-leaks.mjs · line 12

```
const terms = [process.env.LEAK_DENYLIST ?? '', existsSync('.denylist') ? readFileSync('.denylist',  
'utf8') : '']  
  .join('\n')  
  .split(/[,\n]/)  
  .map((t) => t.trim().toLowerCase())  
  .filter((t) => t && !t.startsWith('#'));
```

The denylist is never committed, because a committed list would itself leak the names. It comes from a repository secret in CI and a git-ignored `.denylist` locally, and the script prints only where a match is, never which term matched.

11. Explaining it: questions and answers

Why TypeScript and not Python, like many agentic-testing articles?

The articles' Python is orchestration code (LangChain calling a model). Here the orchestration is the agents themselves, defined as prompt files, so the only code is tests and the kit. Playwright's agent tooling (its test MCP server) is Node-only, and the type checker catches agent mistakes for free: a page-object method or fixture an agent invents fails `tsc` before a person reviews it.

Why not let tests self-heal at runtime?

Because a test that rewrites itself to pass can hide a real bug: if "Log in" quietly becomes "Delete account", a healed test passes. Here repairs are proposals: classified, checked by code, re-run and reviewed in a pull request.

What stops the healer from cheating anyway?

The guard, which is code, not a prompt. It rejects edits outside the test folders, skips, fixed waits, swallowed errors, fewer assertions, weaker matchers and loosened values, and the rejected changes are undone. It was rehearsed with a deliberately dishonest healer and rejected every cheat.

How do you know the AI-written tests are any good?

Three ways. A person reviews each plan before tests are generated. The coverage report shows which rules have tests. The mutation runner breaks each rule on purpose and checks the covering tests fail; a test that can't fail is caught there.

Why do tests create their own users?

Independence. No shared seed accounts means tests can run in parallel against one backend and never fail because another test changed "their" data. Cleanup is automatic, even when a test fails.

Why sign in through the API instead of the form?

Speed and isolation: only sign-in tests depend on the login form, so a broken form fails a few tests instead of all of them.

Why accessible locators only?

Tests that find elements the way assistive technology does catch accessibility regressions for free, survive styling changes, and are what agents generate most reliably.

What happens to a mutation run that gets killed?

The journal on disk holds the original file contents; the next run restores them before doing anything else. Database breaks restore from definitions read at apply time.

How do you know the Supabase adapter works outside one project?

The repository has its own Supabase example (`examples/supabase-app`): a notes app with row-level security, an onboarding function and constraints. CI starts its local stack and runs its tests, coverage and seven database and app breaks on every push.

What would you improve next?

A combined dashboard of coverage and mutation results over time, and a planner prompt that also proposes the matching breaks for `e2e/mutations.ts`, so every new rule arrives with a test and a proof that the test can fail.

12. Glossary

Term	Meaning here
Adapter	The app-specific code behind the fixtures: make, sign in and delete users; seed data
Agent	A Claude Code subagent defined in <code>.claude/agents/*.md</code>
Context	YAML describing what a feature should do; the source of truth
Covers	The test annotation naming the context ids a test covers, e.g. <code>errands#private-lists</code>
Fixture	A value a Playwright test asks for by name, with setup and teardown
Guard	<code>checkDiff()</code> : the code that rejects dishonest repairs
Healing	Repairing tests that are wrong while reporting apps that are wrong
MCP	Model Context Protocol; here the server that gives agents a browser and a test runner
Mutation	A deliberate break of the app, to prove the covering tests fail
Seed	A tiny test that sets up a known starting state for an agent
Web-first assertion	An <code>expect(locator)</code> assertion that retries until it passes or times out